12

Hooks globais e Entrega

O material 11 protegeu o domínio de cada agent. Falta proteger a sessão como um todo — e definir o que significa "pronto". Hooks globais são os gates de qualidade que rodam independente de qual agent está ativo; a entrega é o critério final: qualquer pessoa clona o repositório e reproduz o ambiente completo, com agents, skills, hooks e rules funcionando.

A IDEIA CENTRAL DESTE MATERIAL

O evento mais poderoso do encerramento é o Stop: ele pode bloquear o fim da sessão com exit 2, forçando o agent a continuar até os critérios passarem. "Terminei" deixa de ser opinião do modelo e vira resultado de teste.

1 Hooks globais de encerramento

Vivem em `.claude/settings.json` e valem para a sessão inteira. A estrutura para três gates de Stop:

```
{
  "hooks": {
    "Stop": [
      { "hooks": [ { "type": "command",
          "command": ".claude/hooks/global/run-tests.sh" } ] },
      { "hooks": [ { "type": "command",
          "command": ".claude/hooks/global/check-plan.sh" } ] },
      { "hooks": [ { "type": "command",
          "command": ".claude/hooks/global/generate-report.sh" } ] }
    ]
  }
}
```

Gate 1 — a suíte de testes bloqueia o encerramento

```
#!/bin/bash
# .claude/hooks/global/run-tests.sh
INPUT=$(cat)
[ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ] && exit 0

echo "Rodando suite completa de testes..." >&2
pytest tests/ -q 2>&1
if [ $? -ne 0 ]; then
  echo "BLOQUEADO: testes falhando. Corrija antes de encerrar." >&2
  exit 2   # agent não encerra — volta a trabalhar
fi
exit 0
```

Gate 2 — tasks abertas no PLAN.md geram aviso

```
#!/bin/bash
# .claude/hooks/global/check-plan.sh
INPUT=$(cat)
[ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ] && exit 0

OPEN=$(grep -c '^\- \[ \]' PLAN.md 2>/dev/null || echo 0)
if [ "$OPEN" -gt 0 ]; then
  echo "AVISO: $OPEN tasks ainda abertas no PLAN.md:" >&2
  grep '^\- \[ \]' PLAN.md >&2
fi
exit 0   # avisa, mas não bloqueia
```

Repare na diferença deliberada: teste falhando **bloqueia** (exit 2); task aberta **avisa** (exit 0). Nem todo gate precisa ser um muro — mas todo gate precisa ser visível.

OpenCode — plugin equivalente

```
// .opencode/plugin/session-close.ts
import type { Plugin } from "@opencode-ai/plugin"

export const SessionClose: Plugin = async ({ client, $ }) => {
  return {
    event: async ({ event }) => {
      if (event.type === "session.idle") {
        const tests = await $`pytest tests/ -q`.quiet().nothrow()
        if (tests.exitCode !== 0) {
          await client.session.prompt({
            text: `Testes falhando antes de encerrar:\n` +
                  `${tests.stderr}\nCorrija antes de concluir.`,
            noReply: false
          })
        }
      }
    }
  }
}
```

2 Entrega: o que precisa estar versionado

O projeto está pronto quando o ambiente inteiro viaja no repositório:

```
.claude/
├── agents/
│   ├── code-reviewer.md
│   ├── backend-agent.md
│   ├── frontend-agent.md
│   └── qa-agent.md
├── skills/
│   └── nome-da-skill/SKILL.md
├── hooks/
│   ├── global/    (run-tests.sh, check-plan.sh, generate-report.sh)
│   └── backend/   (validate-scope.sh, run-module-tests.sh)
├── rules/
│   ├── seguranca.md
│   └── backend/api.md
└── settings.json
CLAUDE.md
PLAN.md
SPEC.md
```

3 Critérios de entrega

- ☐ O projeto passou pelo review (material 10) **sem itens BLOQUEANTES**

- ☐ Os hooks globais de encerramento **não bloquearam** a última sessão
- ☐ Os comandos de setup do CLAUDE.md funcionam **do zero, numa máquina limpa**
- ☐ O PLAN.md tem a seção de **próximos passos** atualizada com o que ficou pendente

4 Erros mais comuns

ERRO	CORREÇÃO
Scripts de hook sem permissão de execução após o clone	`chmod +x .claude/hooks/**/*.sh` antes da primeira sessão — sem isso, falha silenciosa.
`.env` no repositório	O .claudeignore protege o agent de *ler*; quem impede o *commit* é o .gitignore. Confirme os dois antes do push (material 06).
PLAN.md sem "próximos passos"	O dia seguinte começa sem saber onde o anterior parou. Último ato de toda sessão: atualizar o estado no PLAN.md.

▸ **COLA PARA FIXAR**

- Hooks globais = gates da **sessão**; hooks por agent = contrato do **domínio** (material 11).
- **Stop com exit 2 bloqueia o encerramento**: o agent só "termina" quando os testes deixam.
- Gates podem **bloquear** (teste falhando) ou **avisar** (task aberta) — escolha deliberadamente.
- Todo Stop hook checa `stop_hook_active` para não entrar em loop.
- Entrega = repositório que **reproduz o ambiente do zero**: agents, skills, hooks, rules, settings versionados.
- Sem bloqueantes no review + setup funcionando em máquina limpa + próximos passos no PLAN.md = **pronto**.